

F O R T I G A T E N S E 4

Lab 10 — Security Profiles

01. Inspection Modes

Before any security profile is configured, one decision governs everything that follows: whether the FortiGate inspects traffic in flow-based or proxy-based mode. The setting exists in two places — on the firewall policy and on each profile — and the two must agree. A great deal of the troubleshooting in this lab traces back to that single requirement.

► 1.1 How each mode handles traffic

Flow-based inspection examines packets as they pass through the FortiGate. Traffic is scanned in a single pass, against the same pattern databases, without being held. Because nothing is buffered, latency is low and throughput is high. The cost is context: the engine sees a stream of packets rather than a complete object, so decisions that require the whole file — an archive that must be opened, a document whose true type is only knowable once assembled — cannot be made reliably.

Proxy-based inspection terminates the client connection at the FortiGate, reconstructs the complete object, inspects it, and only then forwards it. The FortiGate behaves as a man-in-the-middle by design. This costs memory and adds latency, and it changes the client's experience of a block: because the FortiGate owns the session, it can return a replacement page explaining what happened rather than simply dropping the transfer.

Property	Flow-based	Proxy-based
Buffering	None — packet by packet	Whole object reconstructed
Latency	Lower	Higher
Memory use	Lower	Higher
Block feedback	Session reset; often no page	Replacement page returned to the client
Archive scanning	Limited	Full — archives can be opened and scanned
True file type detection	Limited	Reliable
Typical use	High-throughput perimeter, latency-sensitive paths	Where inspection depth matters more than throughput

► 1.2 Which profiles use which mode

Not every profile is available in both modes, and several behave differently depending on which mode is in force. The profile dialogue itself advertises this: options restricted to proxy mode carry a red P badge, and they grey out the moment the feature set is switched to flow-based.

Profile	Flow-based	Proxy-based	Notes
AntiVirus	Yes	Yes	Proxy mode adds archive scanning and the replacement page. Flow mode resets the session instead.
Web Filter	Yes	Yes	Proxy mode adds web content filtering, search-keyword logging, and per-user quota enforcement.
File Filter	Yes	Yes	Proxy mode is required for archive inspection and for reliable true-type matching.
Application Control	Yes	Yes	Signature matching is effectively flow-based regardless; the engine runs in the IPS process.
Intrusion Prevention	Yes	Yes	Always flow-based internally — IPS does not buffer.
DNS Filter	Yes	Yes	Operates on DNS queries, so object reconstruction is not relevant.
Video Filter	No	Yes	Proxy only.
DLP	No	Yes	Requires the full object.
SSL/SSH Inspection	Yes	Yes	Certificate inspection works in both. Full inspection is what the other profiles depend on for encrypted traffic.

Within AntiVirus specifically, the APT protection options show the same split. **Content Disarm and Reconstruction** — which strips active content from documents and rebuilds a safe copy — is proxy-only, because disarming a file requires possessing all of it.

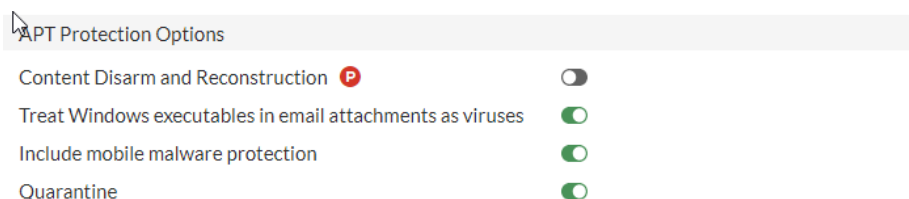


Figure 1 — APT protection options. Content Disarm and Reconstruction carries the proxy-only badge.

► 1.3 Why the choice matters operationally

The two modes are not simply fast and slow versions of the same thing. They answer different questions. Flow-based inspection asks whether anything in this stream matches a known pattern. Proxy-based inspection asks what this object actually is, and then whether it is permitted.

A profile built for one mode and attached to a policy running the other is accepted by FortiOS without an error, and appears correctly attached in the policy list, but does not enforce. This is documented as a finding in section 7 because it cost a full test cycle in this lab.

The policy-level mode is set with `set inspection-mode proxy` or `set inspection-mode flow` under `config firewall policy`. The profile-level mode is the Feature set control in each profile. For this lab both were set to proxy throughout.

02. Topology and Test Environment

The lab deliberately keeps everything inside the EVE-NG environment and the local host. Nothing depends on internet reachability, which matters for two reasons: the FortiGuard content databases on this evaluation VM are either empty or a decade old, and the lab needs to be reproducible by anyone reading it.

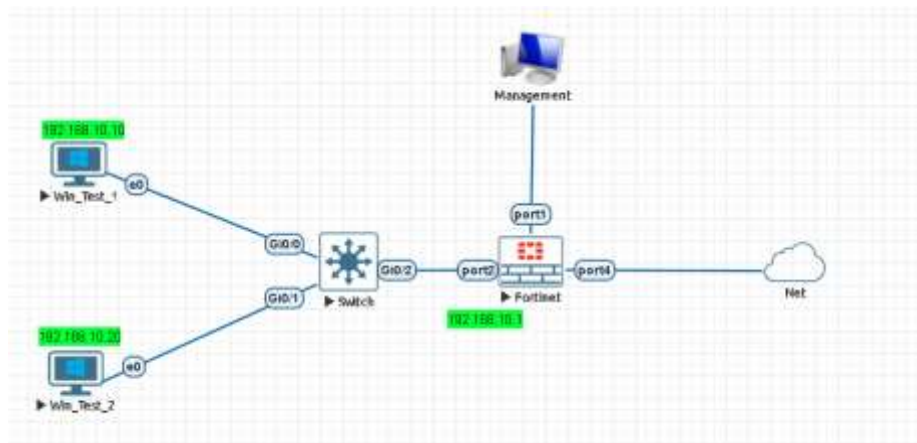


Figure 2 — Lab topology. Two Windows clients behind a switch into port2; port4 faces the host network where the test web server runs.

Element	Address	Role
FortiGate port2	192.168.10.1/24	Client-side gateway
FortiGate port1	Management subnet	GUI and CLI access
FortiGate port4	Host network	Path to the web server
Win_Test_1	192.168.10.10	Inspected client — all profiles attached to its policy
Win_Test_2	192.168.10.20	Control client — no profiles attached
Web server	172.16.28.147	Host PC serving the test files over HTTP

► 2.1 Why two clients

A single inspected client can only ever show that a transfer failed. It cannot show why. Running an identical request from a second client whose policy carries no profiles turns every result into a controlled comparison: same file, same server, same moment, one variable.

Without the control client, a blocked download and a broken web server are indistinguishable from the client's point of view. Each test in this lab was therefore run from both machines.

► 2.2 Connectivity check before inspection

Reachability was confirmed from both clients before any profile existed. This is not a formality: every subsequent result is interpreted as the effect of a profile, and that interpretation is only valid if the path was known good beforehand.

```
C:\Users\user>ping 172.16.28.147

Pinging 172.16.28.147 with 32 bytes of data:
Reply from 172.16.28.147: bytes=32 time=3ms TTL=127
Reply from 172.16.28.147: bytes=32 time=2ms TTL=127
Reply from 172.16.28.147: bytes=32 time=4ms TTL=127
Reply from 172.16.28.147: bytes=32 time=3ms TTL=127
```

Figure 3 — ICMP reachability from a client to the web server host, before any profile was attached.

► 2.3 The test web server

Python's built-in HTTP server is used because it needs no installation, serves whatever is placed in its working directory, and logs every request with its response code — which later proved useful in distinguishing a block from a missing file.

```
C:\Windows\System32>mkdir fgt-lab && cd fgt-lab
C:\Windows\System32\fgt-lab>python3 -m http.server 80
Serving HTTP on :: port 80 (http://[::]:80/) ...
```

Figure 4 — The web server started from the test directory on the host.

```
mkdir fgt-lab && cd fgt-lab
python3 -m http.server 80
```

03. AntiVirus

AntiVirus inspects files traversing the firewall and matches them against the FortiGuard virus signature database. On a licensed appliance the database updates continuously; on this evaluation VM it does not update at all, which raised the question of whether the section was testable before any configuration was written.

► 3.1 The EICAR test file

The EICAR test file is the standard safe trigger for antivirus testing — a 68-byte printable ASCII string, agreed by the anti-malware industry, that every engine is expected to detect and that does nothing whatsoever if executed. It exists precisely so that detection can be demonstrated without handling real malware.

```
X50!P%AP[4\pZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*
```

Figure 5 — The EICAR test string. The file contains this single line and nothing else.



Figure 6 — The server's directory listing confirming the file is published and retrievable.

Local antivirus will delete the file before the firewall ever sees it

Microsoft Defender recognises the EICAR string and quarantines it on write, and again on read. The practical symptom is a 404 from the web server and an apparently inexplicable failure — the file was created, it is gone, and nothing on the FortiGate is at fault.

A folder exclusion on the web server's working directory resolves it. The exclusion belongs on the **server** only; the clients and the FortiGate are deliberately left as they are, because the FortiGate is the engine under test.

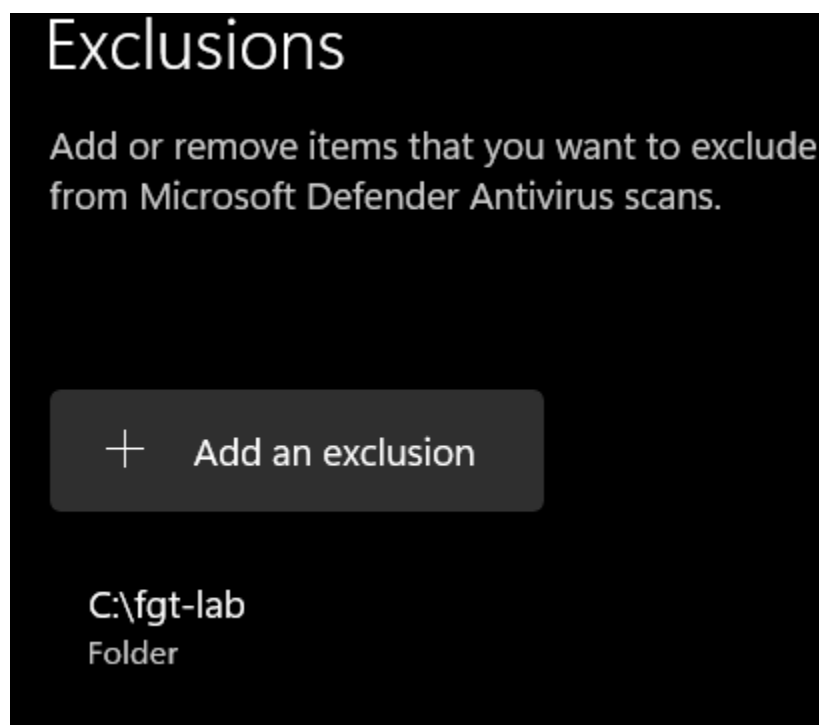


Figure 7 — Defender folder exclusion applied to the web server's working directory.

► 3.2 Baseline — retrieving the file with no profile attached

With the policies in place but no profiles attached, the file was requested from the client. The server recorded the request and returned it successfully, establishing that the path, the policy, and the server were all functioning before inspection was introduced.

```
C:\Users\user>curl -O http://172.16.28.147/eicar.com.txt
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           %             0         0             0      0 0:00:01 --:--:-- 0:00:01 14838
```

Figure 8 — Retrieving the test file from the client before any profile is attached.

eicar.com.txt	10/3/2026 12:13 AM	Text Document	1 KB
NTUSER.DAT	5/23/2021 7:20 PM	DAT File	1,024 KB

Figure 9 — The retrieved file on the client.

```
172.16.31.110 - - [03/Oct/2026 00:40:00] "GET /eicar.com.txt HTTP/1.1" 200 -
```

Figure 10 — The web server log showing the request answered with HTTP 200.

► 3.3 Profile configuration

Figure 11 — The AntiVirus profile. Action set to Block, feature set to proxy-based, HTTP inspection enabled.

Setting	Value	Reason
AntiVirus scan	Block	Monitor logs a detection but still delivers the file. Block is what the test needs to demonstrate.
Feature set	Proxy-based	Matches the policy inspection mode and produces the replacement page rather than a silent reset.
Inspected protocols	HTTP	The only protocol exercised in this topology. Enabling the others would be configuration without evidence.

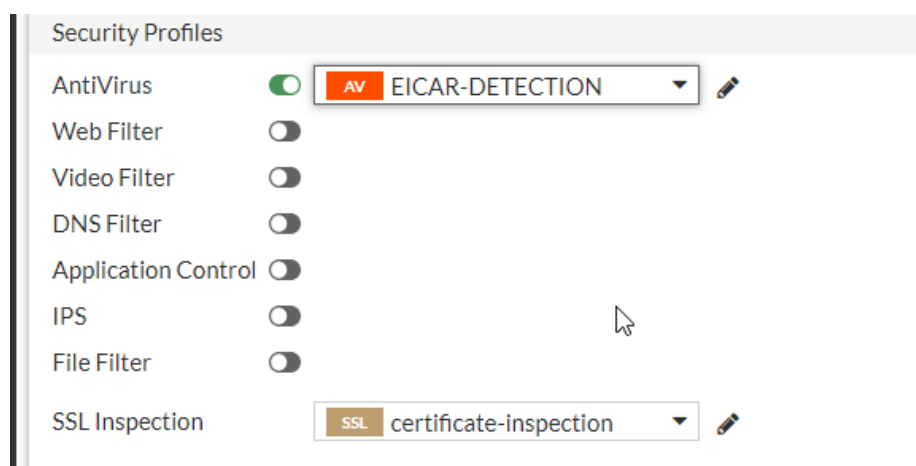


Figure 12 — The profile attached to the inspected client's policy. The control client's policy is left untouched.

► 3.4 Result

The same request was repeated from the inspected client. The transfer completed, but returned 5395 bytes rather than the 68 bytes of the EICAR file — the size of the FortiGate replacement page, not the file.

```
C:\Users\user>curl -O http://172.16.28.147/eicar.com.txt
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total     Spent    Left     Speed
100  5395  100  5395    0     0   5395      0  0:00:01 --:--:--  0:00:01 112k
```

Figure 13 — The retrieved object is 5395 bytes, far larger than the 68-byte test file.

Opening the retrieved object confirms what was actually delivered.



Figure 14 — The FortiGate block page, naming the file and the detected signature.

The page names the signature as `EICAR_TEST_FILE` and links to the FortiGuard encyclopedia entry for it. The log records the same event with the source address, the file name, and the signature.

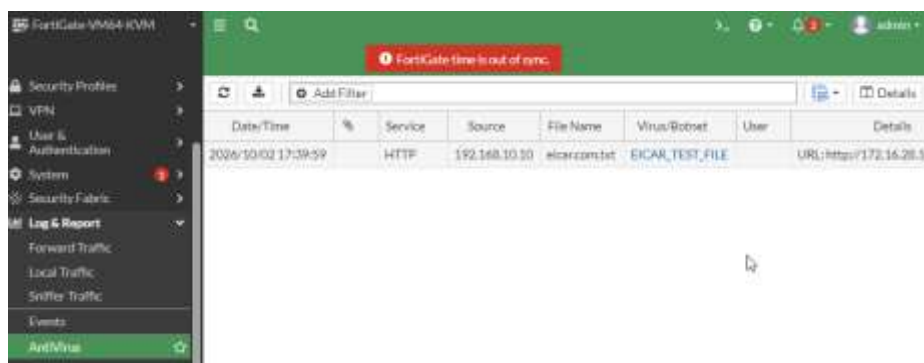


Figure 15 — The AntiVirus log entry for the blocked transfer.

04. File Filter

File Filter blocks transfers by what a file is rather than by what it is called. It is the strongest section in this lab, for a reason worth stating plainly: it depends on no FortiGuard database at all. Where AntiVirus needs signatures and Web Filter needs URL categories, file type identification is structural.

► 4.1 True file type versus extension

Extensions are metadata chosen by whoever named the file. They can be changed by anyone, cost nothing to change, and are routinely changed to evade filters. A filter that trusts them is trivially bypassed.

FortiOS instead reads the leading bytes of the file — its magic number — and identifies the format from its structure. A Windows executable begins with `4D 5A` ("MZ"), a PDF with `%PDF`, a ZIP archive with `PK`. Renaming the file does not alter those bytes, so the identification holds.

► 4.2 Profile and rule

The screenshot shows the 'Create New File Filter Rule' configuration window in FortiGate. The rule is named 'test_blocking'. The 'Comments' field is empty. Under 'Protocols', CIFS, FTP, HTTP, IMAP, POP3, and SMTP are selected. The 'Traffic' section has 'Both' selected. Under 'Match Files', 'Password-protected only' is disabled. Under 'File types', bat, exe, and msi are selected. The 'Action' section shows 'Monitor' and 'Block' options, with 'Block' being the active selection.

Figure 16 — The File Filter rule. Executable types selected, action set to Block.

Edit File Filter Profile

Name: test-blocking

Comments: Write a comment... 0/255

Scan archive contents: ☒

Feature set: ☐ Flow-based ☒ Proxy-based

Rules

Rule	Comments	Traffic	Protocols	Match Files	Action	File Types
test_blocking		Both	HTTP FTP +4	Any	Block	bat exe msi

Figure 17 — The File Filter profile with archive scanning enabled and the feature set corrected to proxy-based.

Setting	Value	Reason
File types	bat exe msi	Executable formats — the category most commonly disguised by renaming.
Traffic direction	Both	Catches uploads as well as downloads. Only the download direction is exercised here.
Protocols	HTTP and others	Only HTTP is tested in this topology; the remaining protocols are configured but not exercised.
Scan archive contents	Enabled	Allows a blocked type to be detected inside an archive rather than only when transferred directly.
Feature set	Proxy-based	Required to match the policy mode. See section 7.1.

Security Profiles

AntiVirus: ☒ AV EICAR-DETECTION

Web Filter: ☐

Video Filter: ☐

DNS Filter: ☐

Application Control: ☐

IPS: ☐

File Filter: ☒ FF test-blocking

SSL Inspection: ☐ SSL certificate-inspection

Figure 18 — Both AntiVirus and File Filter attached to the inspected client's policy.

► 4.3 Test files

Three files were prepared on the server, chosen so that extension and true type disagree in a controlled way.

```
Microsoft Windows [Version 10.0.26200.9457]
(c) Microsoft Corporation. All rights reserved.

C:\fgt-lab>copy C:\Windows\System32\notepad.exe report.txt
1 file(s) copied.

C:\fgt-lab>copy C:\Windows\System32\notepad.exe notepad.exe
1 file(s) copied.

C:\fgt-lab>echo plain text > genuine.txt
```

Figure 19 — Creating the three test files on the web server.

```
copy C:\Windows\System32\notepad.exe report.txt
copy C:\Windows\System32\notepad.exe notepad.exe
echo plain text > genuine.txt
```

File	Extension claims	Actual content	Expected result
notepad.exe	Executable	Executable	Blocked
report.txt	Text	Executable	Blocked
genuine.txt	Text	Text	Delivered

report.txt is the file that matters. It is named as text, requested as text, and served by the web server with a text content type. Only its internal structure gives it away.



Figure 20 — The server's directory listing showing all four published files

► 4.4 Result

```

C:\Users\user>curl -O http://172.16.28.147/notepad.exe
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left     Speed
100  5035  100  5035    0     0   5035      0  0:00:01 --:--:-- 0:00:01 104k

C:\Users\user>curl -O http://172.16.28.147/report.txt
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left     Speed
100  5033  100  5033    0     0   5033      0  0:00:01 --:--:-- 0:00:01 53542

C:\Users\user>curl -O http://172.16.28.147/genuine.txt
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left     Speed
100    13  100    13    0     0     13      0  0:00:01 --:--:-- 0:00:01 419

```

Figure 21 — All three files requested in sequence from the inspected client.

File requested	Bytes returned	What was actually delivered
notepad.exe	5035	FortiGate block page
report.txt	5033	FortiGate block page
genuine.txt	13	The file itself

The two-byte difference between the blocked responses is the file name embedded in the replacement page — `notepad.exe` is two characters longer than `report.txt`. It is not evidence of different handling.

File name	Date and time	File type	Size
genuine.txt	10/3/2026 2:14 AM	Text Document	1 KB
notepad.exe	10/3/2026 2:14 AM	Application	5 KB
NTUSER.DAT	5/23/2021 7:20 PM	DAT File	1,024 KB
report.txt	10/3/2026 2:14 AM	Text Document	5 KB

Figure 22 — The retrieved objects on the client. Both blocked files are approximately 5 KB; only `genuine.txt` is its true size.

Opening each retrieved object confirms the outcome individually.

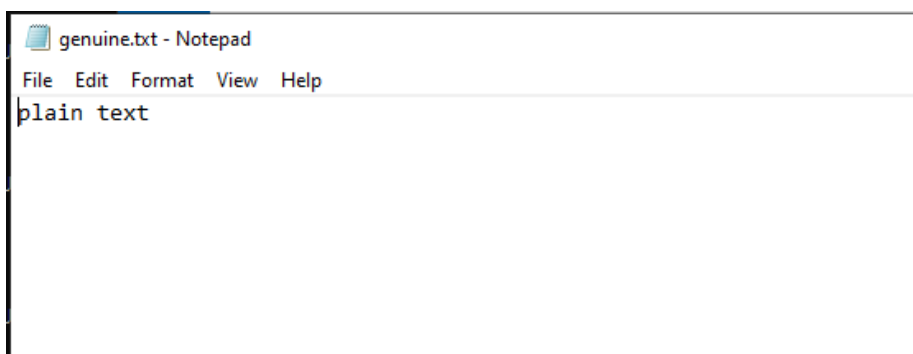


```

report.txt - Notepad
File Edit Format View Help
margin: 1em 1.5em;
}
.flex-container {
display: flex;
}
.flex-container.column {
flex-direction: column;
}
</style>
<title>Attention</title>
</head>
<body><div class="message-container">
<div class="logo"></div>
<h1>Attention</h1>
<p>The file "report.txt" has been blocked due to its file type and/or properties.</p>
<table><tbody>
<tr>
<td>URL</td>
<td>http://172.16.28.147/report.txt</td>
</tr>
</tbody></table>
</div></body>
</html>

```

Figure 23 — report.txt opened on the client: the FortiGate block page, citing file type and properties.



```

genuine.txt - Notepad
File Edit Format View Help
plain text

```

Figure 24 — genuine.txt opened on the client: the original content, delivered intact.

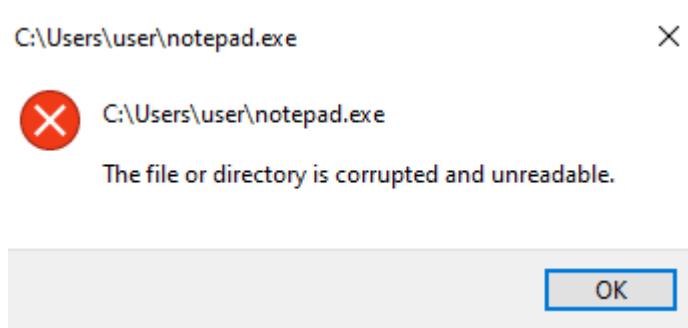
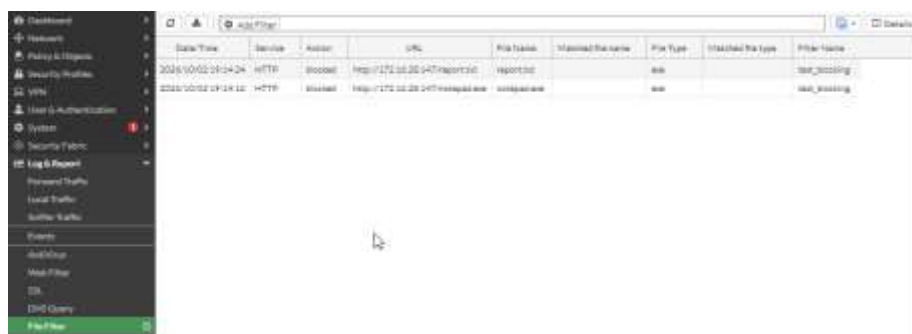


Figure 25 — The object saved as notepad.exe fails to execute — it is an HTML page with an .exe extension, not a binary.

The log attributes both blocks to the rule by name and, critically, records the file type it determined.



Date/Time	Service	Action	URL	File Name	Classified File Name	File Type	Matched File Type	File Name
2023/10/02 09:24:24	HTTP	Blocked	http://172.16.20.147/report.txt	report.txt		exe	not_matching	
2023/10/02 09:24:12	HTTP	Blocked	http://172.16.20.147/realip.exe	realip.exe		exe	not_matching	

Figure 26 — File Filter log. Both entries are identified as type `exe`, including the one named `report.txt`.

The finding

The File Type column reads `exe` for `report.txt`. The FortiGate ignored the extension, the content type the server supplied, and the file name, and identified the object from its leading bytes.

Set against `genuine.txt` passing through the same policy in the same second, this is a controlled demonstration that the match is structural. An extension-based filter configured identically would have delivered the executable and blocked nothing.

05. Web Filter

Web Filter controls HTTP and HTTPS requests. In its normal deployment it matches requests against FortiGuard URL categories — roughly ninety classifications maintained as a live service. That service requires an active contract, and the category database on this VM is absent.

► 5.1 Static URL Filter as the substitute

Rather than omit the section, it was built using the Static URL Filter: a locally defined list evaluated by the same engine, on the same traffic, producing the same replacement page and the same log type. What changes is the source of the verdict, not the enforcement path.

	FortiGuard categories	Static URL Filter
Verdict source	Live FortiGuard rating service	Local list on the appliance
Licence required	Yes	No
Scale	Entire web, by category	Only what is explicitly listed
Maintenance	Maintained by Fortinet	Maintained by the administrator
Enforcement path	Identical	Identical

Static entries are evaluated **before** category lookup, which is why they are also the correct mechanism for exceptions on a licensed appliance — an allow entry for a site miscategorised by FortiGuard, or a block entry for a site permitted by its category but prohibited by local policy.

► 5.2 Configuration

The screenshot shows the 'New URL Filter' configuration window. It includes the following fields and options:

- URL:** 172.16.28.147/blocked/
- Type:** Simple (selected), Regular Expression, Wildcard
- Action:** Exempt, Block (selected), Allow, Monitor
- Status:** Enable (selected), Disable

Figure 27 — The URL filter entry. Simple match type, action Block, status Enabled.

Match type	Behaviour
Simple	Substring match against the requested URL. Direct and readable; the right default.
Wildcard	Permits * as a placeholder, for example 172.16.28.147/blocked/*.
Regular Expression	Full pattern matching. Powerful, and correspondingly easy to write a pattern that matches more than intended.

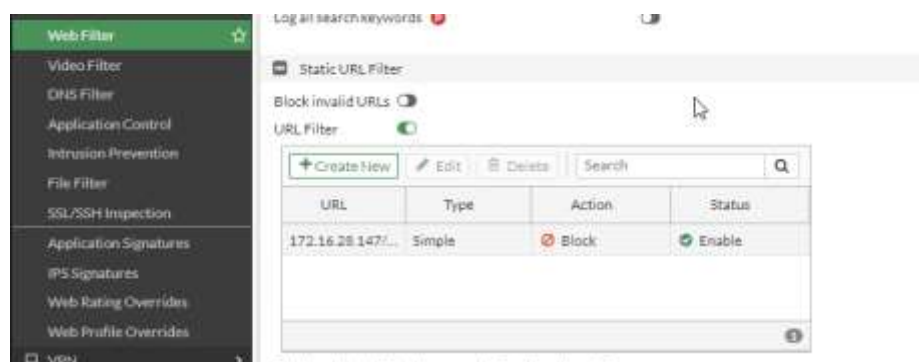


Figure 28 — The Static URL Filter section of the Web Filter profile with the entry enabled.

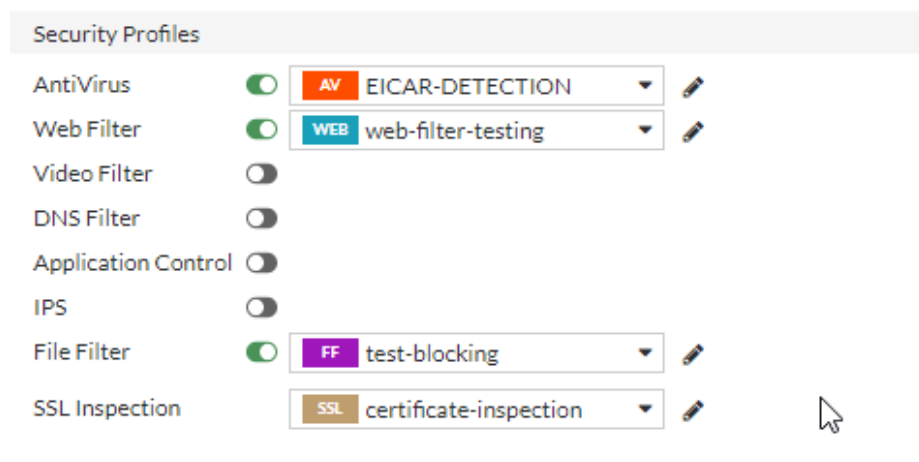


Figure 29 — All three profiles now attached to the inspected client's policy.

► 5.3 Result

The blocked path was requested from the inspected client with `curl -L`, so that the redirect the web server issues for a directory path is followed rather than terminating the request early.

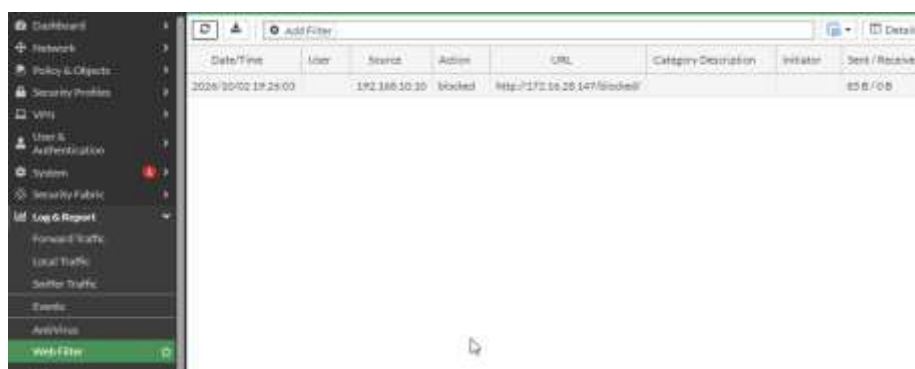
```

</head>
<body><div class="message-container">
<div class="logo"></div>
<h1>FortiGuard Intrusion Prevention - Access Blocked</h1>
<h3>Web Page Blockad</h3>
<p>The page you have requested has been blocked because the URL is banned.</p>
<table><tbody>
<tr>
<td>URL</td>
<td>http://172.16.28.147/blocked/</td>
</tr>
<tr>
<td>Description</td>
<td></td>
</tr>
<tr>
<td>URL Source</td>
<td>Local URLfilter Block</td>
</tr>
</tbody></table>
<p></p>
</div></body>
</html>

```

Figure 30 — The returned object: the FortiGate block page, attributing the verdict to the local URL filter.

The page states the reason as a banned URL and names the source as Local URLfilter Block — distinguishing it from a category verdict, which matters when both mechanisms are in use on a production appliance.



Date/Time	User	Source	Action	URL	Category/Description	Initiator	Sent / Received
2024/10/02 19:28:03		192.168.10.10	blocked	http://172.16.28.147/blocked/			85 B / 0 B

Figure 31 — The Web Filter log entry for the blocked request.

06. Findings

► 6.1 A feature-set mismatch disables a profile silently

Symptom	File Filter profile attached to the policy and visible in the policy list. All three test files, including an executable with its true extension, downloaded without interference. No log entries, no error, no warning anywhere in the GUI.
Cause	The profile was created with the feature set left at flow-based while the policy ran <code>inspection-mode proxy</code> . FortiOS permits the attachment and renders it normally, but the engine does not run.
Fix	Set the profile feature set to proxy-based so that it matches the policy, then verify with <code>show firewall policy <id></code> that both the inspection mode and the profile reference are present.
Rule	A profile that is attached is not necessarily a profile that is running. Verification must come from a test transfer and a log entry, never from the policy list.

This is the single most consequential behaviour in the lab. On a production firewall it produces a policy that passes visual review, survives an audit of the configuration, and inspects nothing.

► 6.2 A block does not always reach the client as a page

The first Web Filter test returned zero bytes and was initially read as a failure. The request had been issued without a trailing slash, which the web server answered with an HTTP 301 redirect; `curl` does not follow redirects unless instructed, so the client saw an empty body and stopped.

The FortiGate log, however, recorded the block against the redirected URL. The enforcement was real and the client's view of it was empty. Where the two disagree, the log is authoritative — the client observes only what reached it, while the appliance records what it decided.

► 6.3 The endpoint's own antivirus interferes with the test

Microsoft Defender quarantined the EICAR file on the web server repeatedly, producing a 404 that was indistinguishable at the client from a transfer failure. The resolution was a folder exclusion scoped to the server's working directory only.

The general point is about test design rather than this file in particular: **when more than one security control sits in the path, a negative result does not identify which one acted.** Each control outside the one under test must be accounted for before its result can be trusted — which is also the argument for the control client used throughout this lab.

07. Licence and Database Constraints

This FortiGate runs under an evaluation licence with no FortiGuard contract. The content databases were examined before the lab was designed, and the scope of each section was decided by what they contained.

Database	State on this VM	Effect on the lab
AntiVirus engine	Present, shipped with firmware	EICAR detection functional — section 3 built as a live test
Virus definitions	1.00000, no update history	Real-world coverage absent; no claim made about it
IPS engine and attack definitions	Present, dated 2015	Not exercised in this lab
Application definitions	Present, dated 2015	Not exercised in this lab
FortiGuard web categories	Absent	Section 5 built on the Static URL Filter instead
FortiGuard DNS categories	Absent	DNS Filter not covered
Botnet domains, Internet Service DB	Empty	Not exercised
File type identification	Not database-dependent	Section 4 unaffected — the strongest section as a result

The constraint is recorded rather than worked around. A lab that quietly omits the sections it cannot run is less useful than one that names them and explains why, and the reasoning above is the same reasoning an engineer applies when scoping what a given appliance can actually enforce.

o8. Summary

Section	Outcome	Evidence
Inspection modes	Flow and proxy behaviour documented, with the per-profile mapping	Proxy-only badges in the profile dialogue
AntiVirus	Blocked	Block page naming EICAR_TEST_FILE; AntiVirus log entry; 5395 bytes returned in place of 68
File Filter — true extension	Blocked	5035 bytes returned; log entry typed exe
File Filter — renamed executable	Blocked	5033 bytes returned; log typed exe despite the .txt name
File Filter — genuine text file	Delivered	13 bytes returned; content intact on the client
Web Filter — static URL	Blocked	Block page citing Local URLfilter Block; Web Filter log at 85 B / 0 B
Control client	Unaffected throughout	Every file retrieved successfully on the policy carrying no profiles
Feature-set mismatch	Documented finding	Profile attached, no enforcement, no error
DNS Filter, Application Control, IPS	Out of scope	Database state recorded in section 7